S20 - Módulo 1. Actividad didáctica 2-M1

Universidad Digital de Antioquia

Ingeniería de Software y Datos
Simulación

YAHTZEE

Aplicación gráfica y simulación Monte Carlo
Ley de los Grandes Números

José del Carmen Pineda Alarcón

25 de octubre de 2026

1. Introducción

El objetivo del proyecto es construir un juego de Yahtzee en Python con interfaz gráfica y una herramienta de simulación estadística. El juego utiliza cinco dados de seis caras y las trece categorías habituales de puntuación. La interfaz muestra el estado de la partida y permite conservar dados entre lanzamientos. El módulo de simulación repite lanzamientos de un dado y representa gráficamente cómo la frecuencia acumulada de la cara 1 se aproxima a 1/6.

La solución se organiza en tres archivos principales: backend.py contiene las reglas, el estado de la partida y el simulador; frontend.py implementa la interfaz con tkinter; main.py inicializa la aplicación. La dependencia para crear la gráfica es matplotlib.

2. Metodología: USA, MODIFICA y CREA

USA ? Usar y comprender. Ejecutar la aplicación, reconocer los controles y observar el recorrido de los datos: los dados generan valores, el evaluador calcula puntos y la interfaz presenta el resultado. Esta etapa permite comprender las reglas modeladas antes de extenderlas.

MODIFICA ? Adaptar la solución. Ajustar el flujo de turnos a la solicitud del proyecto: primero el Jugador 1 completa sus trece rondas y después juega el Jugador 2. Se incorporan conservación de dados, reinicio confirmado, actualización automática de subtotales y una sugerencia de puntuación. Al concluir el tercer lanzamiento, la aplicación selecciona automáticamente la categoría disponible con mayor puntuación inmediata.

CREA ? Construir y comprobar. Separar el programa en módulos, implementar las trece categorías, crear una simulación Monte Carlo configurable entre 100 y 500,000 lanzamientos, generar la gráfica de convergencia e integrar los controles en tkinter. Se comprueba la lógica con ejecuciones que revisan puntuación, límite de lanzamientos, secuencia de turnos y generación de la gráfica.

3. Solución implementada

Backend. La clase Dado genera caras uniformemente con `random.randint(1, 6)`. EvaluadorYahtzee calcula Unos, Doses, Treses, Cuatros, Cincos, Seises, Trío, Póker, Full House, Escalera menor, Escalera mayor, Yahtzee y Chance. JuegoYahtzee mantiene cinco dados, los dados conservados, tres lanzamientos por turno, trece categorías por jugador y los puntajes. La bonificación superior de 35 puntos se concede al alcanzar al menos 63 puntos en las seis categorías numéricas.

Frontend. La ventana de tkinter presenta los dados, la ronda, el jugador activo, los lanzamientos restantes, la hoja de puntos, el subtotal y el total. El control Nueva partida solicita confirmación antes de restablecer el estado. El control Terminar turno y anotar permite cerrar el turno antes del tercer lanzamiento.

Decisión sobre puntuación automática. Para satisfacer el flujo solicitado, el sistema anota la categoría sin usar que dé el mayor puntaje en ese momento. Si varias categorías empatan, se elige la primera según el orden de la hoja. Esta regla es una simplificación de la estrategia: maximiza el puntaje inmediato de esa tirada, pero no calcula una estrategia óptima para el total de la partida y puede gastar una categoría que sería más útil después.

Alcance de Monte Carlo. La simulación estima lanzamientos individuales de un dado, no simula partidas completas ni la estrategia de puntuación. Para cada n calcula la frecuencia acumulada f(n) = X(n)/n, donde X(n) es el número de resultados iguales a 1. La probabilidad teórica es p = 1/6. El error estándar teórico de la frecuencia es sqrt(p(1-p)/n); por ello, la variabilidad típica decrece al aumentar n.

4. Resultados

Para producir resultados repetibles se fijó la semilla aleatoria en 20261025 y se ejecutaron 100,000 lanzamientos. La aplicación guarda la gráfica como convergencia_montecarlo.png.

Lanzamientos (n)	Frecuencia de 1	Error absoluto respecto a 1/6	Error estándar teórico
100	0.190000	0.023333	0.037268
1,000	0.159000	0.007667	0.011785
10,000	0.164700	0.001967	0.003727
100,000	0.166790	0.000123	0.001179

En 100,000 lanzamientos, la frecuencia fue 0.166790 (16.679 %); el valor teórico es 0.166667 (16.667 %). La diferencia absoluta fue 0.000123, menor que el error estándar teórico aproximado de 0.001179. La sucesión de frecuencias no tiene que acercarse de forma monótona: puede oscilar, pero las fluctuaciones suelen reducirse al acumular más observaciones.

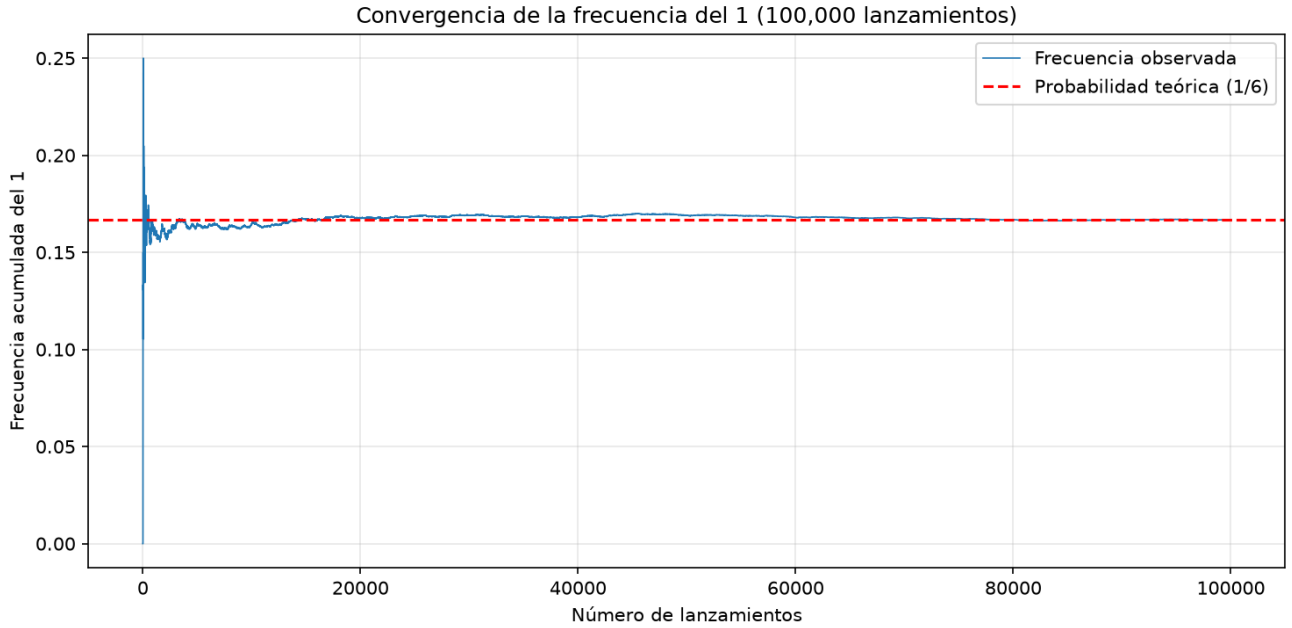


Figura 1. Frecuencia acumulada de la cara 1 frente a la probabilidad teórica 1/6. Simulación reproducible de 100,000 lanzamientos.

5. Conclusiones y limitaciones

La implementación integra la lógica de Yahtzee, una interfaz gráfica y una demostración experimental de la Ley de los Grandes Números. En la corrida documentada, la frecuencia observada se aproximó a la probabilidad 1/6 al aumentar el número de lanzamientos. Este resultado es evidencia experimental coherente con la teoría, no una demostración matemática; otras corridas pueden producir frecuencias distintas por azar.

La selección automática de categoría responde a la preferencia del flujo actual, pero no equivale a una estrategia óptima ni al modo tradicional en que una persona elige el casillero. La simulación solo estudia una cara de un dado; una extensión futura podría analizar las frecuencias de las seis caras o simular partidas completas. La interpretación de USA, MODIFICA y CREA debe revisarse si se obtiene la guía oficial del curso.

6. Ejecución y referencias

Desde PowerShell, dentro de la carpeta del proyecto:

```
py -m pip install -r requirements.txt
py main.py
```

Referencias bibliográficas

[1] Ross, S. M. (2014). A First Course in Probability (9.ª ed.). Pearson. Referencia para probabilidad y convergencia de frecuencias.

[2] Python Software Foundation. random ? Generate pseudo-random numbers. https://docs.python.org/3/library/random.html

[3] Python Software Foundation. tkinter ? Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html

[4] Matplotlib Development Team. Matplotlib documentation. https://matplotlib.org/stable/

[5] Código de la solución: backend.py, frontend.py y main.py, incluidos en el proyecto.

Video de presentación

El siguiente enlace dirige a la presentación del proyecto Yahtzee y su simulación Monte Carlo.

https://youtube.com/live/nwZVAd_pAxg?feature=share

Presentación compartida por YouTube Live.

Repositorio del proyecto

https://github.com/jdcpa1978-IUD/Juego_Yahtzee_Montecarlo_Pineda_jose